



Foundations of Programming

Assignment No. : 5

PROBLEM STATEMENT:

Write a program in C to perform basic matrix operations such as:

1. Addition of two matrices
2. Saddle point of a matrix
3. Inverse of a matrix
4. Magic square of a matrix

OBJECTIVE:

- To understand matrix representation in C using two-dimensional arrays.
- To implement various matrix operations using C programming.
- To develop logical thinking for solving matrix-based problems.

THEORY:

Matrix operations are widely used in engineering and scientific applications. Addition of matrices involves adding corresponding elements. A saddle point is an element that is the minimum in its row and maximum in its column. The inverse of a matrix exists only for square matrices with non-zero determinant. A magic square is a square matrix in which the sums of elements in each row, column, and diagonals are equal. **(ADD Example for 1. Addition of two matrices**

2. Saddle point of a matrix

3. Inverse of a matrix

4. Magic square of a matrix

)

PLATFORM:

Linux

ALGORITHM:

1. Start
2. Read the order of the matrix
3. Read elements of the matrices
4. Perform matrix addition
5. Check for saddle point

6. Find inverse of the matrix (if determinant $\neq 0$)
7. Check whether the matrix is a magic square
8. Display results
9. Stop

CODE :

```

1  #include <stdio.h>
2
3  #define MAX 10
4
5  // Function for Matrix Addition
6  void addMatrices(int a[MAX][MAX], int b[MAX][MAX], int r, int c) {
7      int sum[MAX][MAX], i, j;
8
9      printf("\nResultant Matrix:\n");
10     for(i = 0; i < r; i++) {
11         for(j = 0; j < c; j++) {
12             sum[i][j] = a[i][j] + b[i][j];
13             printf("%d ", sum[i][j]);
14         }
15         printf("\n");
16     }
17 }
18
19 // Function to find Saddle Point
20 void saddlePoint(int a[MAX][MAX], int r, int c) {
21     int i, j, k, min, col, found = 0;
22
23     for(i = 0; i < r; i++) {
24         min = a[i][0];
25         col = 0;
26
27         for(j = 1; j < c; j++) {
28             if(a[i][j] < min) {
29                 min = a[i][j];
30                 col = j;
31             }
32         }
33
34         for(k = 0; k < r; k++) {
35             if(a[k][col] > min)
36                 break;
37         }

```

```

38
39         if(k == r) {
40             printf("Saddle Point = %d\n", min);
41             found = 1;
42         }
43     }
44
45     if(!found)
46         printf("No Saddle Point found\n");
47 }
48
49 // Function for Inverse of 2x2 Matrix
50 void inverseMatrix(float a[2][2]) {
51     float det;
52
53     det = a[0][0]*a[1][1] - a[0][1]*a[1][0];
54
55     if(det == 0) {
56         printf("Inverse not possible (Determinant = 0)\n");
57         return;
58     }
59
60     printf("Inverse Matrix:\n");
61     printf("%.2f %.2f\n",  a[1][1]/det, -a[0][1]/det);
62     printf("%.2f %.2f\n", -a[1][0]/det,  a[0][0]/det);
63 }
64
65 // Function to check Magic Square
66 void magicSquare(int a[MAX][MAX], int n) {
67     int i, j, sum = 0, diag1 = 0, diag2 = 0, flag = 1;
68
69     for(j = 0; j < n; j++)
70         sum += a[0][j];
71

```

```

72     for(i = 0; i < n; i++) {
73         int rowSum = 0, colSum = 0;
74
75         for(j = 0; j < n; j++) {
76             rowSum += a[i][j];
77             colSum += a[j][i];
78         }
79
80         if(rowSum != sum || colSum != sum) {
81             flag = 0;
82             break;
83         }
84     }
85
86     for(i = 0; i < n; i++) {
87         diag1 += a[i][i];
88         diag2 += a[i][n-i-1];
89     }
90
91     if(diag1 != sum || diag2 != sum)
92         flag = 0;
93
94     if(flag)
95         printf("It is a Magic Square\n");
96     else
97         printf("Not a Magic Square\n");
98 }
99
100 int main() {
101     int choice;
102
103     do {
104         printf("\n--- MATRIX OPERATIONS ---\n");
105         printf("1. Addition\n2. Saddle Point\n3. Inverse (2x2)\n4. Magic Square\n5. Exit\n");

```

```
106     printf("Enter choice: ");
107     scanf("%d", &choice);
108
109     if(choice == 1) {
110         int a[MAX][MAX], b[MAX][MAX], r, c, i, j;
111
112         printf("Enter rows and columns: ");
113         scanf("%d %d", &r, &c);
114
115         printf("Enter Matrix A:\n");
116         for(i=0;i<r;i++)
117             for(j=0;j<c;j++)
118                 scanf("%d",&a[i][j]);
119
120         printf("Enter Matrix B:\n");
121         for(i=0;i<r;i++)
122             for(j=0;j<c;j++)
123                 scanf("%d",&b[i][j]);
124
125         addMatrices(a, b, r, c);
126     }
127
128     else if(choice == 2) {
129         int a[MAX][MAX], r, c, i, j;
130
131         printf("Enter rows and columns: ");
132         scanf("%d %d", &r, &c);
133
134         printf("Enter Matrix:\n");
135         for(i=0;i<r;i++)
136             for(j=0;j<c;j++)
137                 scanf("%d",&a[i][j]);
138
139         saddlePoint(a, r, c);
```

```

140     }
141
142     else if(choice == 3) {
143         float a[2][2];
144         int i, j;
145
146         printf("Enter 2x2 Matrix:\n");
147         for(i=0;i<2;i++)
148             for(j=0;j<2;j++)
149                 scanf("%f",&a[i][j]);
150
151         inverseMatrix(a);
152     }
153
154     else if(choice == 4) {
155         int a[MAX][MAX], n, i, j;
156
157         printf("Enter order of square matrix: ");
158         scanf("%d", &n);
159
160         printf("Enter Matrix:\n");
161         for(i=0;i<n;i++)
162             for(j=0;j<n;j++)
163                 scanf("%d",&a[i][j]);
164
165         magicSquare(a, n);
166     }
167
168     } while(choice != 5);
169
170     return 0;
171 }

```


INPUT:

Matrix elements entered by the user. *Type Input also paste Screen Shot of Input*

OUTPUT:

Result of matrix addition, saddle point, inverse, and magic square verification. *Type Output also paste Screen Shot of Output*

```
--- MATRIX OPERATIONS ---
1. Addition
2. Saddle Point
3. Inverse (2x2)
4. Magic Square
5. Exit
Enter choice: 1
Enter rows and columns: 2
2
Enter Matrix A:
1 2 3 4
Enter Matrix B:
2 3 4 5

Resultant Matrix:
3 5
7 9
```

CONCLUSION:

Thus, we have learned how to perform basic matrix operations using C programming and understood their practical applications.

FAQs:

1. What is a saddle point in a matrix?
2. When does a matrix have an inverse?
3. What is a magic square?